

分类学 AI Agent 建设指南

从个人分类 workflow、脚本与经验规则，构建可持续迭代的 Taxonomy Research Agent

结构化项目设计手册 · v0.1

核心目标

不是让 AI 替代分类学家做分类决策，而是把多年积累的工作流、术语体系、脚本、证据规则和质量控制固化为一个可复现、可审核、可持续更新的研究系统。

整理日期：2026-10-06

0. 一页看懂：这个项目到底是什么

TaxonomyAgent 的本质不是一个“会聊天的分类学机器人”，而是一个“**专家知识系统 + 结构化分类数据库 + 你的科研脚本 + 文献证据库 + AI 编排层**”的组合。



第一版最重要的原则

先做一个“可靠的单 Agent + 5~6 个工具”，不要一开始就做多 Agent、知识图谱、自动微调或自动修改核心规则。分类学的价值首先来自数据质量、证据追踪和专家判断，而不是 Agent 数量。

阶段	你要实现的能力	是否建议现在做
v0.1	读取性状矩阵、比较物种、调用地图脚本、生成证据化结果	立即做
v0.2	加入 PDF 文献库和基于文献的问答 (RAG)	第二步
v0.3	自动更新检索表、坐标 QC、名称核对	第三步
v0.4	自动生成 Diagnosis / Description / Remarks 草稿	第四步
v0.5	反馈记录、回归测试、版本化规则更新	必须做
v1.0	多项目、多属、多 Agent 协作与成熟 UI	后期

目录

1. 为什么这个方向值得做
2. 现有项目：哪些真正值得参考
3. 推荐总体架构
4. 分类学数据标准与性状本体
5. v0.1：第一版到底要实现什么
6. 项目目录与核心文件
7. 把你已有脚本变成 Agent Tools
8. 文献知识库与证据追踪
9. 检索表、分布图与稿件自动化
10. 持续更新：怎样让 Agent 真正越来越好
11. 技术选型与不建议现在做的事情
12. 90 天开发路线图
- 附录 A. 推荐 GitHub / 项目清单
- 附录 B. 可直接使用的规则模板与数据表结构

1. 为什么这个方向值得做

分类学研究中，有大量工作高度重复、规则明确、但又需要专家监督。例如：文献中性状抽取、种间比较、经纬度整理、分布图生成、检索表更新、术语统一、Diagnosis 与 Remarks 的初稿组织。AI 很适合承担这些“高重复 + 高信息量 + 可验证”的环节。

1.1 AI 最适合承担什么

任务	AI / Agent 适合做的部分	必须由分类学家负责的部分
文献整理	抽取名称、性状、模式产地、分布、引文和页码	核对原始文献与解释语境
性状矩阵	标准化术语、发现缺失值、比较物种	决定哪些性状具有真正分类学价值
检索表	根据矩阵寻找分裂性状、生成候选 couplets、自动测试路径	确认性状稳定、可观察且不会误导
分布图	坐标清洗、格式转换、调用 R/Python/GIS 脚本	核实 locality 与错误坐标
稿件初稿	组织 Description、Diagnosis、Remarks 和图注	最终分类判断与学术责任
质量控制	检查 unknown/absent 混淆、引用缺失、术语不一致	决定是否接受修改

最重要的工作原则

Observation → Structured data → Comparison → Interpretation → Writing。不要走 “Observation → AI → Manuscript” 的捷径。中间必须有结构化数据层。

2. 现有项目：哪些真正值得参考

目前没有一个成熟开源项目能够直接完成“个人分类学工作流 + 文献库 + 性状矩阵 + 检索表 + 分布图 + 稿件”的完整闭环。因此最合理的做法不是寻找一个现成项目直接替代你的工作，而是从成熟项目中分别借鉴数据模型、文献抽取、标准和 Agent 编排思想。

项目	主要用途	你应该借鉴什么	推荐
TaxonWorks	分类学与生物多样性综合工作台	Taxon、name、specimen、	★★★★★

项目	主要用途	你应该借鉴什么	推荐
		citation、collecting event、descriptor 等数据关系；非常适合参考数据库结构	
Plazi / GoldenGATE Imagine	分类学 PDF treatment 抽取	从 PDF 中识别 taxonomic treatment、名称、material、occurrence、reference 的流程	★★★★★
TDWG SDD	结构化分类描述与性状数据标准	Character / state / description / key 的机器可读模型	★★★★★
Darwin Core	生物多样性 occurrence / specimen 标准	scientificName、decimalLatitude、locality、typeStatus、basisOfRecord 等字段	★★★★★
PaperQA2	科学论文 RAG	文献库问答、证据片段与 inline citation 思路	★★★★★
pygbif / taxize	GBIF 和分类名称数据接口	名称核对、occurrence 下载、外部数据库查询	★★★★☆
OpenAI Agents SDK	Agent 编排框架	tools、sessions、handoffs、human-in-the-loop、tracing	★★★★★
LangGraph	复杂有状态 Agent 工作流	分支、暂停人工审核、恢复执行；适合后期复杂化	★★★★☆
Specify 7	自然历史标本与 collections 管理	标本、采集事件、collections 数据设计	★★★★☆☆
OpenBiodiv	生物多样性知识图谱	后期构建 taxon-publication-occurrence-trait 知识网络时参考	★★★★☆☆

TaxonWorks: <https://github.com/SpeciesFileGroup/taxonworks>

GoldenGATE Imagine: <https://github.com/plazi/GoldenGATE-Imagine>

Plazi ontologies: <https://github.com/plazi/ontologies>

TDWG SDD: <https://github.com/tdwg/sdd>

Darwin Core: <https://github.com/tdwg/dwc>

PaperQA2: <https://github.com/Future-House/paper-qa>

pygbif: <https://github.com/gbif/pygbif>

taxize: <https://github.com/ropensci/taxize>

OpenAI Agents SDK: <https://github.com/openai/openai-agents-python>

LangGraph: <https://github.com/langchain-ai/langgraph>

不要直接照搬 TaxonWorks

TaxonWorks 很强，但它是大型协作平台。你的第一版只需要借鉴它的数据实体关系，不应该把整个系统 fork 后再大改。这样会把大量时间耗在与你当前目标无关的 Web 开发和数据库复杂性上。

3. 推荐总体架构

3.1 建议的四层结构

Layer 1 专家规则层

- 你的 taxonomy SOP
- 性状判定规则
- 名称与命名学注意事项
- 写作与证据规则

Layer 2 结构化数据层

- specimen matrix
- species character matrix
- coordinates
- literature metadata
- evidence records

Layer 3 工具层

- Python / R / GIS 脚本
- PDF / RAG
- GBIF / WoRMS / Catalogue of Life 等接口
- Excel / SQLite

Layer 4 Agent 编排层

- 理解任务
- 选择工具
- 汇总证据
- 输出候选结论
- 请求人工审核

设计原则：LLM 不做“事实数据库”

物种性状、标本记录、坐标、文献来源应该放在结构化文件或数据库中。LLM 负责推理、调度工具和生成语言，不应该成为唯一的数据存储。

4. 分类学数据标准与性状本体

4.1 Species character matrix：第一版的数据核心

第一版可以从 CSV / Excel 开始，不需要马上上 PostgreSQL 或知识图谱。最重要的是字段稳定、含义明确，并且严格区分缺失与缺如。

species	branchiae_start	branchiae_end	anal_papillae	black_spicules	lateral_groove
O. yangi	5	28	10	present	present
O. species_A	6	26	8	unknown	present
O. species_B	5	30	12	absent	absent

强制规则

unknown ≠ absent。文献没有描述某个性状，只能记为 unknown，不能自动当作 absent。这个规则应该进入 Agent 的核心 instructions，并写进自动测试。

4.2 Character ontology：统一“同个性状的不同写法”

不同论文可能用不同表达描述同一性状。Agent 应先标准化到内部字段，再在输出稿件时转回自然语言。

文献写法：

- branchiae start at chaetiger 5
- first branchia on chaetiger 5
- branchiae commencing from chaetiger V

内部标准字段：

```
character_id: branchiae_first_chaetiger
value: 5
unit: chaetiger
source: literature / observation
```

4.3 Evidence schema：每个结论都要能追溯

字段	示例	用途
species	O. yangi	物种
character	black_spicules	标准化性状 ID
value	present	性状状态

字段	示例	用途
source_type	OBSERVED / LITERATURE	证据类型
reference	Jimi et al. 2021	来源文献
page	p. 8	页码 / treatment 位置
specimen	PM-001-PM-020	直接观察时的标本范围
confidence	high / medium / low	证据质量
notes	visible through body wall	上下文和限制

5. v0.1：第一版到底要实现什么

第一版不要追求“全自动 taxonomy”。只做下面六个能力，而且每个能力都必须可验证。

1. 读取 species character matrix (CSV / Excel) 。
2. 比较一个目标种与所有 congeners，输出 unique / shared / missing / potentially diagnostic characters。
3. 读取坐标表，做基础 QC，然后调用你现有的 distribution map R/Python 脚本。
4. 读取本地文献元数据或少量 PDF，并能返回来源与页码。
5. 根据结构化性状生成 Diagnosis / Description / Remarks 候选初稿，但必须标注证据状态。
6. 运行质量检查：unknown 是否被误当 absent、引用是否缺失、是否出现未经证据支持的 taxonomic assertion。

v0.1 成功标准

你输入一个 species 名称后，系统能够用你自己的矩阵和脚本完成比较、地图和候选稿件，而且每个关键结论能告诉你“证据来自哪里”。这比做一个会说很多话的 Agent 更重要。

6. 项目目录与核心文件

```
TaxonomyAgent/  
├── agent/  
│   ├── main.py  
│   ├── instructions.md  
│   └── tools.py  
├── knowledge/  
│   ├── literature/  
│   ├── taxonomy_rules/  
│   └── terminology/  
├── data/  
│   ├── specimens/  
│   ├── characters/  
│   ├── coordinates/  
│   └── taxonomy/  
├── scripts/  
│   ├── distribution_map.R  
│   ├── character_matrix.py  
│   ├── key_generator.py  
│   └── coordinate_check.py  
├── projects/  
│   └── ophelia_revision/  
├── outputs/  
│   ├── maps/  
│   ├── tables/  
│   └── manuscripts/  
├── feedback/  
│   ├── corrections.jsonl  
│   └── proposed_rules.md  
└── tests/  
    └── benchmark_questions.json
```

6.1 instructions.md：你的“数字化分类学 SOP”

```
# Role
You are a taxonomic research assistant.
The human taxonomist is the final authority.

# Evidence policy
Every taxonomic statement must be classified as:
OBSERVED / LITERATURE_REPORTED / INFERRED / UNKNOWN

Never convert UNKNOWN into ABSENT.
Never invent character states.
Never invent references.

# Comparison
Prefer original descriptions and revisions.
Distinguish missing information from character absence.
Prefer discrete and stable morphological characters.
Treat measurements affected by preservation cautiously.
Geographic separation alone is not diagnostic.

# Diagnosis
Call a character diagnostic only when comparative coverage supports it.
If congeners are insufficiently surveyed, use cautious wording such as
"potentially taxonomically informative".

# Dichotomous key
Prefer stable, easy-to-observe, discrete characters.
Every taxon must resolve to exactly one terminal.
```

7. 把你已有脚本变成 Agent Tools

你已经积累的 R / Python / GIS 脚本是这个 Agent 最重要的资产之一。最好的做法不是让 AI 重写，而是给现有脚本加一个稳定接口。

7.1 示例：调用 distribution_map.R

```
import subprocess

def run_distribution_map(csv_file: str) -> str:
    result = subprocess.run(
        ["Rscript", "scripts/distribution_map.R", csv_file],
        capture_output=True,
        text=True
    )

    if result.returncode != 0:
        return f"Map generation failed: {result.stderr}"

    return result.stdout
```

Agent 的职责只是判断“现在应该调用地图工具”；真正的制图逻辑仍由你成熟的脚本控制。因此你的配色、底图、比例尺、投影、输出分辨率和期刊格式都可以保持稳定。

7.2 最小 Agent 示例

```
from agents import Agent, Runner
from pathlib import Path

instructions = Path("agent/instructions.md").read_text(encoding="utf-8")

taxonomy_agent = Agent(
    name="Taxonomy Research Assistant",
    instructions=instructions,
    tools=[read_character_matrix, run_distribution_map]
)

result = Runner.run_sync(
    taxonomy_agent,
    """
    Read data/characters/ophelia_matrix.csv.
    Compare O. yangi against all other species.
    Return unique states, shared states, missing data,
    and potentially diagnostic characters.
    Do not draft manuscript text yet.
    """
)

print(result.final_output)
```

建议

Agent 先输出结构化分析，再写文章。把“比较”和“写作”拆成两个阶段，可以显著降低模型为了写得顺而自行补充事实的风险。

8. 文献知识库与证据追踪

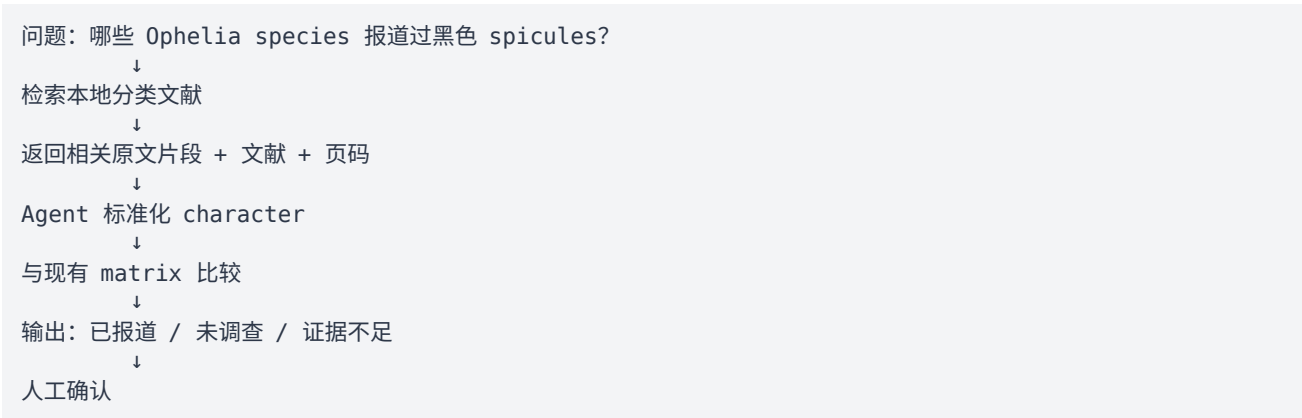
8.1 第一阶段：本地 PDF + 文献元数据

```
knowledge/literature/
├── Jimi_2021.pdf
├── Hartman_1950.pdf
├── Day_1967.pdf
└── literature_index.csv
```

literature_index.csv 至少记录：author、year、title、taxon、document type、file path、DOI / URL、是否为 original description / revision / redescription。这样比直接把所有 PDF 扔给模型更可控。

8.2 第二阶段：RAG / PaperQA2 思路

当文献数量变多后，再加入 RAG。目标不是让模型“记住所有论文”，而是让它在回答之前检索与你的问题最相关的原文片段，并把文献、页码或 treatment 一起返回。

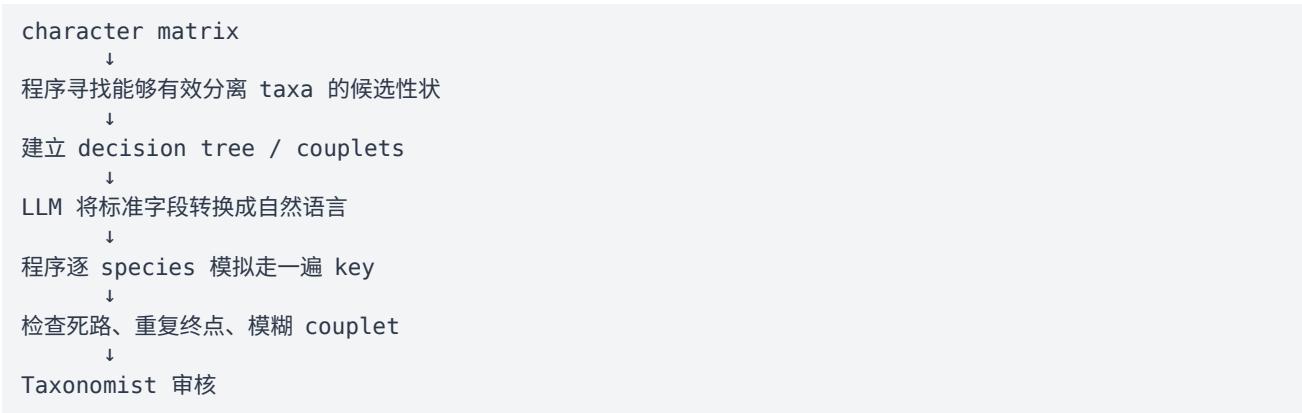


禁止逻辑

“没有检索到”不能自动等于“该物种不存在这个性状”。结果必须能够表达：not reported、not examined、unknown、absent、present。

9. 检索表、分布图与稿件自动化

9.1 检索表：让程序负责逻辑，让 LLM 负责语言



9.2 分布图：必须先做坐标 QC

检查项	自动规则示例
合法范围	latitude ∈ [-90, 90]; longitude ∈ [-180, 180]

检查项	自动规则示例
格式统一	DMS 转 decimal degrees; 明确 E/W、N/S
重复记录	同 species + 坐标 + locality 检测重复
海陆异常	海洋物种点落入明显内陆时标记人工复核
文献来源	每个点保存 source / page / occurrence ID
record 类型	type locality / literature record / new record / uncertain

地图错误的典型来源

很多“地图画错”并不是绘图软件的问题，而是经纬度抄录、E/W 方向、DMS 转换、地点名称与坐标不一致、文献 OCR 等问题。Agent 应把异常点列出来，而不是默默画图。

9.3 稿件生成：按模块，而不是一键写整篇

7. Material examined: 只整理标本信息，不解释。
8. Description: 只根据 observation / structured matrix 描述。
9. Diagnosis: 只使用有足够比较证据的性状。
10. Remarks: 比较近似种，明确哪些信息来自文献、哪些来自直接观察。
11. Discussion: 在前面事实层稳定后再做解释。
12. Consistency check: species name、figure number、character state、measurement、reference 全文交叉检查。

10. 持续更新：怎样让 Agent 真正越来越好

“持续更新”不应该等同于让模型自动修改自己的核心提示词。分类学是证据驱动领域，核心规则必须可审计、可回滚。推荐把每次人工纠正记录成反馈，再由你批准是否进入正式规则。

```
Agent 输出错误
↓
你纠正
↓
feedback/corrections.jsonl
↓
系统生成 candidate rule
↓
feedback/proposed_rules.md
```

```
↓
人工审核
↓
批准后写入 instructions.md / ontology
↓
Git commit
↓
运行 benchmark tests
↓
确认没有 regression
```

```
{
  "task": "species comparison",
  "error": "treated unknown as absent",
  "correction": "unknown must remain unknown",
  "approved_rule": true
}
```

10.1 建一个“分类学考试题库”

```
[
  {
    "question": "Can black spicules currently be considered diagnostic?",
    "expected": "No; comparative coverage among congeners is insufficient."
  },
  {
    "question": "Does missing information mean a character is absent?",
    "expected": "No."
  }
]
```

每次修改 instructions、ontology 或工具后都跑一遍 benchmark。这样你才能知道系统是真的变好，还是只是在某一次聊天里“看起来更聪明”。

11. 技术选型与不建议现在做的事情

11.1 推荐技术栈

模块	第一版建议	理由
Agent	Python + OpenAI Agents SDK	轻量、工具调用清晰、适合把现有脚本包装成 tools
数据表	CSV / Excel + pandas	最容易从现有工作流迁移
结构化数据库	SQLite	单人项目够用，迁移简单
地图与统计	保留现有 R / Python / GIS	不要重写成熟脚本

模块	第一版建议	理由
文献检索	本地 PDF → 后期 PaperQA2 / RAG	先保证来源可追溯
外部数据	pygbif / taxize + 其他可靠 API	用于名称和 occurrence 辅助核对
版本管理	Git / GitHub	规则、脚本、ontology、benchmark 都能追踪
后期复杂流程	LangGraph	只有出现大量分支、暂停与恢复需求后再引入

11.2 第一阶段不建议做

- 不要一开始就做 10 个 Agent。
- 不要一开始就用 Neo4j / 知识图谱作为核心数据库。
- 不要马上 fine-tune 模型；先解决数据结构和工具调用。
- 不要让 Agent 自动修改 instructions.md。
- 不要让模型直接决定 synonym、validity、holotype / paratype 等命名学关键结论。
- 不要把“未检索到文献”当作“性状不存在”。
- 不要让模型自行编造 reference、页码或 specimen information。

12. 90 天开发路线图

周期	目标	主要产出
第 1-2 周	整理现有工作流与脚本	instructions.md；脚本清单；标准文件夹；1 个真实 genus 项目
第 3-4 周	建立 character matrix + evidence schema	characters.csv；evidence.csv；术语词典
第 5-6 周	完成 v0.1 Agent	读取矩阵、species comparison、调用地图脚本
第 7-8 周	加入 PDF / 文献检索	literature index；RAG 原型；页码证据
第 9-10 周	检索表与稿件模块	key generator；Diagnosis / Description 分模块生成
第 11-12 周	反馈与测试体系	corrections.jsonl；

周期	目标	主要产出
		benchmark_questions.json；版本 v0.2

最适合的试验项目

选一个你最熟悉、数据最完整、已经有脚本和论文经验的属作为 pilot。不要一开始处理整个动物门。一个属跑通全流程以后，再把数据模式推广到其他类群。

附录 A. 推荐 GitHub / 项目清单

TaxonWorks — 大型 taxonomy workbench；重点看数据实体与工作流，不建议直接 fork 做第一版。

<https://github.com/SpeciesFileGroup/taxonworks>

GoldenGATE Imagine — 分类学 PDF 解析与 treatment 处理。 <https://github.com/plazi/GoldenGATE-Imagine>

Plazi Ontologies — taxonomic treatment / semantic publishing 的 本体参考。

<https://github.com/plazi/ontologies>

TDWG SDD — 结构化描述、character/state、keys 的标准参考。 <https://github.com/tdwg/sdd>

Darwin Core — occurrence、specimen、locality 等 biodiversity data 标准。

<https://github.com/tdwg/dwc>

PaperQA2 / PaperQA — 科学文献检索与基于证据回答。 <https://github.com/Future-House/paper-qa>

pygbif — Python GBIF 客户端。 <https://github.com/gbif/pygbif>

taxize — R 生态中的 taxonomy 数据接口工具。 <https://github.com/ropensci/taxize>

OpenAI Agents SDK — 第一版 Agent 工具调用与编排框架。 <https://github.com/openai/openai-agents-python>

LangGraph — 复杂有状态 workflow，适合后期。 <https://github.com/langchain-ai/langgraph>

说明：开源项目的 API、维护状态和目录结构会变化。真正开始集成某个项目之前，应再次检查其最新版 README、release 与 license。

附录 B. 可直接使用的规则模板与数据表结构

B1. 建议的证据状态枚举

状态	含义	允许用于 diagnosis?
OBSERVED	你或团队直接观察到	通常可以，但仍需比较
LITERATURE_REPORTED	可靠原始文献明确报道	可以作为比较证据
INFERRED	根据上下文推断	原则上不能直接作为诊断性事实
UNKNOWN	没有数据 / 文献未描述 / 未检查	不能
ABSENT	明确检查后未发现	可以，但要记录检查依据

B2. 建议的 corrections.jsonl 记录字段

```
{
  "date": "2026-10-06",
  "project": "ophelia_revision",
  "task": "diagnosis",
  "agent_output": "Black spicules are diagnostic.",
  "problem": "Comparative coverage among congeners is insufficient.",
  "expert_correction": "Use potentially taxonomically informative.",
  "rule_candidate": "Do not call a character diagnostic when congener coverage is incomplete.",
  "approved": true
}
```

B3. 一个最终工作流示例

用户：更新 0. yangi 的 Remarks 和分布图

1. Agent 读取 species matrix
2. Agent 检索与近似种有关的文献证据
3. Agent 输出差异性状表（不写 prose）
4. Taxonomist 审核差异表
5. Agent 调用 coordinate_check.py
6. 异常点人工确认
7. Agent 调用 distribution_map.R
8. Agent 根据已审核差异表生成 Remarks 草稿
9. QC Agent / checker 检查：
 - 是否把 unknown 当 absent
 - 是否有无来源的 assertion
 - species names / figure numbers 是否一致
10. Taxonomist 最终审核并提交

最终建议

把这个项目看作“把你自己的分类学方法软件化”，而不是“训练一个 AI 取代你的判断”。模型可以更换，但你积累的 ontology、rules、scripts、evidence、tests 和 corrections 才是最长期的核心资产。